国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

「Course」
EDHOC on a Rocket RV64 SoC

「Lesson 5」 Secure file transfer and results

Khai-Duy Nguyen

2026/12

Today’s lesson purpose:

- Use the key from the handshake: an encrypted channel that streams a file of any size from one board's SD card to the other's.
- Read the radio and SD diagnostics, and know what to do when a transfer fails.
- Measure the speed yourself, choose the radio or the 3× faster wired link, and reproduce every result of the course.

Code: fpga/sw/export/common (edhoc_app_data, edhoc_rf_link, sd_card, fat32, edhoc_file_xfer)

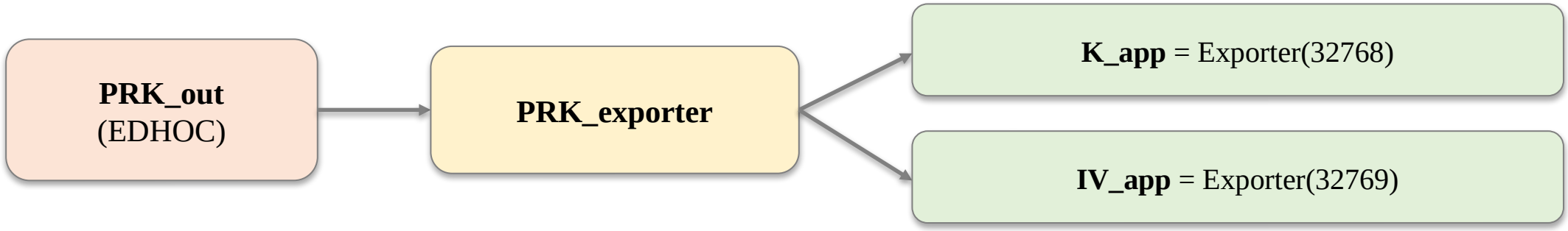
Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Speed: Measure and Choose a Link
6. Choose and Reproduce
7. Practice

Outline

1. **Application Channel**
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Speed: Measure and Choose a Link
6. Choose and Reproduce
7. Practice

1. Application Channel (1/4) Keys for the application



- Same suite, same keys on both boards — nothing secret is sent.
- The AEAD is the suite's **application AEAD**: AES-CCM-16-64-128, AES-128/256-GCM or ChaCha20/Poly1305; key, IV and tag sizes are read at run time, so one code path serves every suite.
- The file demo runs with AuthKEM TBD3: AES-256-GCM, 32-byte key.

1. Application Channel (2/4) One record

Part	Content	Sent?
header (5 B)	direction (0 = A→B, 1 = B→A) ‖ 32-bit sequence number	no — both sides know it
nonce	IV_app XOR header (in the last 5 bytes)	no
ciphertext	AEAD(K_app, nonce, aad = header, plaintext ≤ 206 B)	yes
tag	8 or 16 bytes	yes

common/src/edhoc_app_data.c

```
static void mk_nonce(const struct app_chan *ch, uint8_t *nonce, const uint8_t *hdr)
{
    uint32_t off = ch->iv_len - APP_HDR_LEN;
    memcpy(nonce, ch->base_iv, ch->iv_len);
    for (uint32_t i = 0; i < APP_HDR_LEN; i++)
        nonce[off + i] ^= hdr[i];
}
```

- The two directions never share a nonce.
- A lost, repeated or reordered record fails its tag.

1. Application Channel (3/4) Using it from your code

```
common/include/edhoc_app_data.h
struct app_chan ch;
app_data_init(&ch, suite, &prk_out, is_initiator);    // keys from PRK_out

app_data_send(&ch, buf, len, tx, sock);               // len <= APP_PT_MAX (206)
app_data_recv(&ch, buf, sizeof(buf), &got, rx, sock); // decrypt + verify
```

- *tx* / *rx* are the same transport functions as the handshake (radio or UART).
- Records are at most 206 bytes: 206 + 16-byte tag fills exactly two radio fragments (section 2).
- Every send and receive also adds to an encrypt / decrypt cycle counter — the `PERF encrypt` / `PERF decrypt` lines.

1. Application Channel (4/4) Demo traffic

```
initiator, TBD3
APP: channel up (aead=3 key=32 iv=12 tag=16)
APP: sent    'REQ 0 FROM INITIATOR'
APP: got     'ECHO[REQ 0 FROM INITIATOR]'
APP: sent    'REQ 1 FROM INITIATOR'
APP: got     'ECHO[REQ 1 FROM INITIATOR]'
...
PERF encrypt      89688 cyc 0.001195 s 60 bytes 50209 B/s
APP DATA OK
```

- After every handshake: three requests and echoes both ways prove both keys and both directions.
- Then, depending on the build:
  - `APP_FILE_BYTES=n` — stream n generated bytes (checked by regenerating them)
  - `-DAPP_SD_XFER` — send SEND.BIN card to card (section 4)

Outline

1. Application Channel
2. **Radio Link**
3. SD Card & FAT32
4. End-to-End Transfer
5. Speed: Measure and Choose a Link
6. Choose and Reproduce
7. Practice

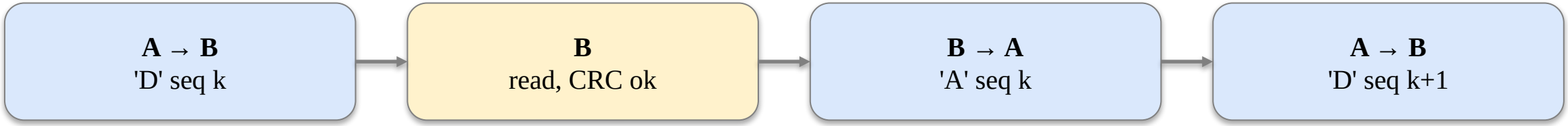
2. Radio Link (1/8) The MRF24J40 and its limits

Property	Value	Consequence
Standard	IEEE 802.15.4, 2.4 GHz, channel 11	shared band with Wi-Fi
Bit rate	250 kbit/s (32 µs per byte)	≈ 4 ms on air for a full frame
Frame	127 B → 116 B payload after the 802.15.4 header	a record needs 2 frames
Receive buffer	one frame	the sender must wait until the receiver read it
Host interface	SPI at 4.7 MHz over PMOD wires	bit errors happen (next slides)

- Hardware ACK: the radio confirms the frame reached the peer's receive buffer.
- It does **not** confirm the peer's software read it — that needs the link's own ACK.

2. Radio Link (2/8) Fragments and stop-and-wait

```
common/include/edhoc_rf_link.h
frame payload = [type][seq][flags][data ... up to 111 B][crc16]
  type  'D' data fragment   flags bit0 = first, bit1 = last of the message
        'A' acknowledgement of fragment seq
```



- The next fragment goes out only after the 'A' for the previous one.
- No 'A' within 30 ms → send the same fragment again; the receiver re-acks a repeat and drops it.
- Simple and safe with a one-frame receive buffer; the price is one round trip per fragment.

2. Radio Link (3/8) One radio frame, byte by byte

Bytes	Field	Written by
2	frame control: data frame, ACK request, PAN ID compression, short addresses	mrf24j40.c
1	802.15.4 sequence number	mrf24j40.c
2	PAN ID 0xABCD	mrf24j40.c
2 + 2	destination / source short address	mrf24j40.c
3	link header: type 'D' or 'A', seq, flags (bit 0 FIRST, bit 1 LAST)	edhoc_rf_link.c
≤ 111	data	edhoc_rf_link.c
2	CRC-16/CCITT over link header + data	edhoc_rf_link.c
2	FCS (802.15.4 CRC)	radio chip

- 9 + 3 + 111 + 2 + 2 = **127 bytes**, the largest 802.15.4 frame.
- Two checksums: the FCS covers the air, the link's CRC-16 also covers the SPI wires on both sides (section 2, (5/8)). The 'A' frame is 16 bytes: header, link header, CRC, FCS.

2. Radio Link (4/8) Splitting and rebuilding a message

```
common/src/edhoc_rf_link.c (shortened)
int32_t rf_link_recv(uint8_t *data, uint32_t cap)
{
    len = recv_frag(f);                    // next 'D' in order, already ACKed
    if (!(f[2] & F_FIRST)) return -2;      // must start a message
    for (;;) {
        copy f[3 .. len) to data + got;    // -2 if it would pass cap
        if (f[2] & F_LAST) return got;     // message complete
        len = recv_frag(f);
        if (f[2] & F_FIRST) return -2;     // a new message inside this one
    }
}
```

Message	Bytes	Fragments
app record: 206 B + 16-B tag	222	111 FIRST, 111 LAST
length / CRC record: 4 B + tag	20	1 (FIRST + LAST)
RFC 9529 message_2	45	1
TBD3 message_4	1,604	14 × 111 + 50 = 15

- seq is 8 bits and wraps at 256: safe, because only one fragment is ever unconfirmed.
- A repeat of the fragment accepted last (its 'A' was lost) is acknowledged again and dropped, so data is never delivered twice.
- The first fragment of the peer's next message also confirms the last one sent (implicit ACK, *take_reply()*).

2. Radio Link (5/8) If a long transfer fails with an AEAD error (113)

You see	XFER: record at <n> failed (113) on board B, after a few thousand records
It means	a record arrived with wrong bytes. On PMOD wires a bit can flip on the SPI link between the CPU and the radio; the radio's own checksum (FCS) is computed after that, so it cannot see it
The code	adds its own CRC-16 to every frame (seal() / poll_frame() in edhoc_rf_link.c): a bad frame gets no 'A' and is sent again
Check	the RF: line at the end of the run: crc_err > 0 with XFER RECV … OK means bad frames were caught and resent
Try it	Exercise 2 removes the CRC and shows the error; restore with git checkout
Remember	check data end to end; a chip's checksum covers only its own part of the path

2. Radio Link (6/8) If one board stops hearing the other

You see	the sender's repeats count climbs, the receiver gets nothing — for more than 2 s
It means	probably a radio register changed by a flipped SPI write (e.g. the channel). Data CRCs cannot fix a radio that is no longer listening
The code	re-initialises its radio after 2 s without progress (heal_if_stuck(), in every waiting loop), keeping all sequence numbers
Check	RF: … reinits=<n>. The responder shows a few while it waits for the initiator to be loaded (one per 2 s) — that is normal
Remember	every wait needs a way out

2. Radio Link (7/8) If both boards keep repeating after the handshake

You see	the pair stops right after EDHOC OK (often with DEBUG=1, whose prints slow one side down); both boards repeat one frame
It means	a radio that holds an unread frame acknowledges nothing; if both sides only transmit, neither ever gets an ACK
The code	take_reply() reads the FIFO between transmit attempts; a finished board keeps answering in transport_idle()
Try it	comment out the take_reply() call in transmit(), build suite 2 method 3 with DEBUG=1, run the pair: it stops in the first echo
Restore	from fpga/sw/export: git checkout common/src/edhoc_rf_link.c, rebuild both images, rerun: PASS
Remember	a link layer must keep listening while it waits

2. Radio Link (8/8) Reading the RF: line

```
Example output — end of a radio run, after APP DATA OK
RF: frames=6 repeats=0 crc_err=0 reinits=0      <- initiator, suite 2 method 3
RF: frames=5 repeats=0 crc_err=0 reinits=0      <- responder
```

Field	Counts	Normal	Worth a look
frames	data fragments sent, repeats included	handshake: tens; 1 MiB: ≈ 10,000	—
repeats	fragments sent again (no 'A' in 30 ms)	0 … a few %	a large share: interference, distance
crc_err	received frames dropped by the CRC-16	0 or a few	many: check the JC wiring
reinits	radio re-initialisations (2 s without progress)	a few on the side that waited first	during a transfer: (6/8)

The same link carries the handshakes of Lessons 3 and 4, so every pair prints this line.

Outline

1. Application Channel
2. Radio Link
3. **SD Card & FAT32**
4. End-to-End Transfer
5. Speed: Measure and Choose a Link
6. Choose and Reproduce
7. Practice

3. SD Card & FAT32 (1/9) What is on the card

Sectors (LBA)	Content	Written by
0	MBR partition table	mksd.sh / sd_prep
2048 … 67583 (partition 1, 32 MiB, raw)	firmware image for SD boot	sd_boot.sh
67584 … end (partition 2, FAT32)	SEND.BIN (board A), RECV.BIN (board B)	your PC / the board

- **End-user flow:** copy any file onto the card as SEND.BIN with a PC, put the card in board A, run — board B writes RECV.BIN on its card.
- FAT32, so any PC reads and writes the files; the boot area stays raw.
- Files up to 4,294,967,294 B: FAT32 sizes are 32-bit and the transfer reserves 0xFFFFFFFF as “no file”. RAM use does not depend on the size.

3. SD Card & FAT32 (2/9) Preparing a card — with or without a card reader

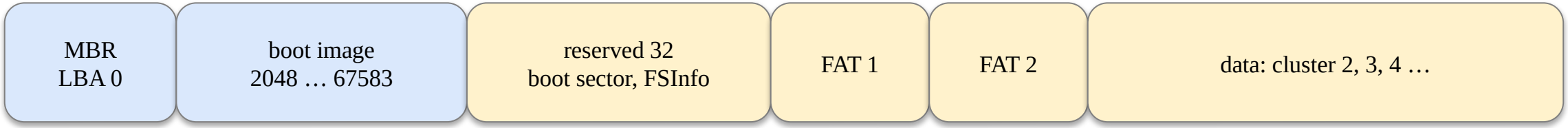
```
Run this — on a PC with a card reader
sudo $REPO/fpga/sw/export/tools/mksd.sh /dev/sdX   # erases the card: MBR + 32 MiB boot + FAT32
# then copy your file as SEND.BIN onto the FAT32 partition (label EDHOC)
```

```
Run this — no card reader: from the board, over JTAG
cd $REPO/fpga/sw/export/tests
make sd_prep EXTRA_CFLAGS="-DSD_PREP_FORMAT -DSD_PREP_SEND=1048576"   # format + 1 MiB test file
../common/board/run_one.sh build/sd_prep.elf 150              # board A: format + SEND.BIN
make sd_prep EXTRA_CFLAGS=-DSD_PREP_FORMAT                     # board B: format only
BOARD=responder ../common/board/run_one.sh build/sd_prep.elf 150
```

```
card: 61067264 sectors (29818 MiB)
formatting...
format OK
SEND.BIN: 1048576 bytes, CRC cffbfb2e
SD PREP OK
```

- *sd_prep* writes the same layout as mksd.sh, plus a test SEND.BIN of known pseudo-random bytes.
- `-DSD_PREP_VERIFY=<bytes>` later checks RECV.BIN on board B: exactly that length, and the same bytes.

3. SD Card & FAT32 (3/9) FAT32 on disk: from cluster to sector



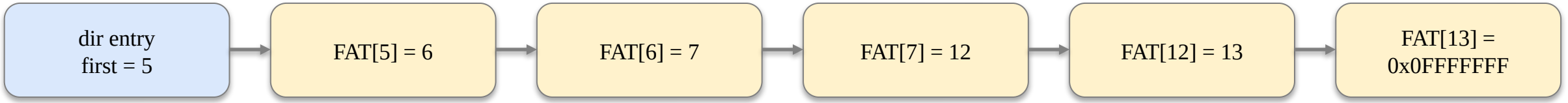

```
fat_lba   = part_lba + reserved                    part_lba = 67584, reserved = 32
data_lba  = fat_lba + nfats * fat_sz               two FAT copies
LBA of cluster c       = data_lba + (c - 2) * spc            spc = sectors per cluster
FAT entry of cluster c = sector fat_lba + c / 128, byte (c % 128) * 4, low 28 bits
                         0 = free,  2 ... = next cluster,  0x0FFFFFFF = end of chain
```

Card (formatted by sd_prep, cluster size as Windows picks it)	spc	Cluster	1 MiB file
board A, 31 GB	32	16 KiB	64 clusters
board B, 3.6 GB	8	4 KiB	256 clusters

All values come from the boot sector at part_lba (bytes 11, 13, 14, 16, 32, 36, 44, 48): fat32_mount() reads them and nothing else.

3. SD Card & FAT32 (4/9) Fragmented files: following the cluster chain

```
root-directory entry, 32 bytes
53 45 4E 44 20 20 20 20 42 49 4E   "SEND    BIN"   name, 8.3 form
20                                 attribute: archive
... bytes 20-21 / 26-27            first cluster, high / low 16 bits
... bytes 28-31                    file size, little endian
```




```
fat32_read()
if (f->pos && f->pos % cb == 0) {        // crossed into the next cluster
    fat_get(f->clus, &f->clus);          // read its FAT entry
    if (!valid(f->clus)) return FAT_ENOFS;
}
```

- Reading follows any chain: a SEND.BIN a PC wrote in pieces works (host test "copy fragmented SEND.BIN"). Cost: one FAT sector per cluster, 63 extra reads for 1 MiB on card A (3 %).
- Writing takes one **contiguous** run of free clusters: no FAT reads while streaming, the chain is written at commit.
- Limit: a card whose free space exists only in pieces answers “no room” (FAT_ENOSPC). Copy the files off and reformat.

3. SD Card & FAT32 (5/9) The SD driver: commands and two timing details

Step	Command
Wake up in SPI mode	≥ 74 clocks, then CMD0 (go idle) — **up to 10 times**
Check voltage / version	CMD8 → echo 0x1AA
Initialise	ACMD41 until ready; CMD58 → block addressing (SDHC)
Read / write one sector	CMD17 / CMD24, 512 B + CRC-16, then wait while busy

- **CMD0 is retried:** right after the card powers up, its first answer can be one junk byte (Lesson 2, “If spi_test shows R1 = 3f”).
- **Busy ends in the middle of a byte:** after a write the card holds the data line low; the line can be released mid-byte (e.g. 0x0f), so the driver waits until a byte reads **0xFF**, not just until it stops reading 0x00.
- See the first one yourself: let CMD0 run once (`t < 1` in *tests/src/periph_spi.c*), pull the card out and in, run spi_test → R1 = 3f FAIL. Restore with `git checkout`.

3. SD Card & FAT32 (6/9) If an SD write is reported as failed

You see	SD: write <sector>: <reason> lines on board B, maybe “XFER: SD write failed” and “not stored”
Reasons	data response 0b = the card saw a CRC error (bits flipped on the wires); 0d = the card could not write; busy timeout = the card never finished
The code	retries each sector up to 3 times. It also reads the card's reply (data response, 05 = accepted) from 4 bytes, because some cards send it a byte late — read one byte too early, a good write looks failed
Check	a few 1 MiB transfers: no SD: lines, XFER RECV … OK, and the read-back with the right length passes (section 4)
If it stays	reseat the card, try another card, reformat it (sd_prep -DSD_PREP_FORMAT)
Remember	log the device's raw reply before deciding who failed

3. SD Card & FAT32 (7/9) One sector over SPI

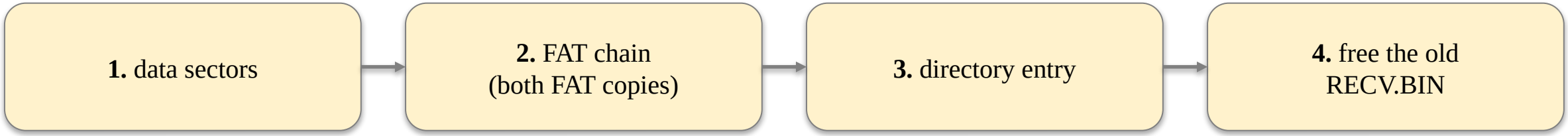
```
reading the FAT32 boot sector of the card
-> 51 00 01 08 00 01          CMD17 read block: 0x40 | 17, LBA 0x00010800 = 67584, CRC byte
<- FF 00                      R1 = 0x00: command accepted
<- FF ... FF FE               start token
<- 512 data bytes, CRC-16     the driver checks the CRC-16

-> 58 <LBA, 4 B> 01           CMD24 write block
<- 00                         R1
-> FE, 512 data bytes, CRC-16
<- xxx0 0101 (0x05)           data response: accepted
<- 00 00 ... 0F FF            busy while the card programs: wait for FF (bug 2)
```

- Every command is 6 bytes: 0x40 | index, a 32-bit argument, CRC-7 + stop bit. In SPI mode only CMD0 (95) and CMD8 (87) need a real CRC.
- SDHC / SDXC cards take the sector number; old SDSC cards a byte address (LBA × 512). The CCS bit of CMD58 says which.
- One sector per command: multi-block commands would save little, since the SD part is 5.7 s (read) and 6.5 s (write) of the 99.6 s transfer.

3. SD Card & FAT32 (8/9) FAT32 in 512 bytes of RAM

- One 512-byte sector buffer for everything: no FAT cache, no cluster buffer — the 64 KiB RAM cannot afford more.
- **Read** SEND.BIN: find it in the root directory (8.3 name), follow its cluster chain sector by sector.
- **Write** RECV.BIN: reserve one contiguous run of free clusters, write the data, then **commit** in a safe order:



- Rejected **before** commit (bad record, wrong CRC, lost link): only free clusters were written, so every file — including the old RECV.BIN — is unchanged.
- Commit itself is several sector writes. An I/O error or power loss **during** commit can leave lost clusters or a half-written chain: run *fsck.fat* on a PC. This is not a power-loss-safe file system.

3. SD Card & FAT32 (9/9) Testing the FAT32 code on the PC

```
cd $REPO/fpga/sw/export/tests; source ~/edhoc-venv/bin/activate; make fat32-host-test
  copy fragmented SEND.BIN over old RECV.BIN   PASS
  RECV.BIN == SEND.BIN (pyfatfs)               PASS
  fsck.fat clean                               PASS
  uncommitted write leaves files unchanged     PASS
  empty SEND.BIN -> new RECV.BIN               PASS
  1000-byte SEND.BIN -> new RECV.BIN           PASS
  format a junk card                           PASS
  MBR: p1 raw at 2048, p2 FAT32 at 67584       PASS
  copy on the formatted card                   PASS
  missing file -> FAT_ENOENT                   PASS
FAT32 HOST TEST: PASS (0 failures)
```

- Same fat32.c, with sd_read / sd_write going to a disk-image file.
- Images made by *mkfs.vfat*, checked by **fsck.fat** and an independent FAT library (pyfatfs).
- Does the test catch bugs? Two deliberately broken builds (no FAT mirroring, old chain not freed) both **FAIL** fsck.

Outline

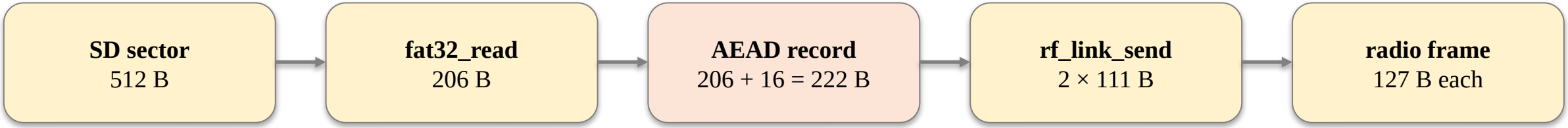
1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. **End-to-End Transfer**
5. Speed: Measure and Choose a Link
6. Choose and Reproduce
7. Practice

4. End-to-End Transfer (1/8) What goes over the air

Record	Plaintext	Purpose
0	4-byte length (0xFFFFFFFF = sender has no SEND.BIN)	receiver reserves space
1 … n	the file, 206 bytes per record, last one shorter	data
n + 1	4-byte CRC-32 of the file	checks the received stream

- Every record is AEAD-protected (section 1) — that is the security. The transfer CRC-32 checks the **received plaintext stream** (SD read → link); only a read-back checks the **stored file** (SD write).
- Neither side holds more than one record and one sector: the same image moves 1 KB or 100 MiB.
- Receiver busy writing the card? The radio repeats fragments until it reads again; the UART link waits for its READY byte (section 5).

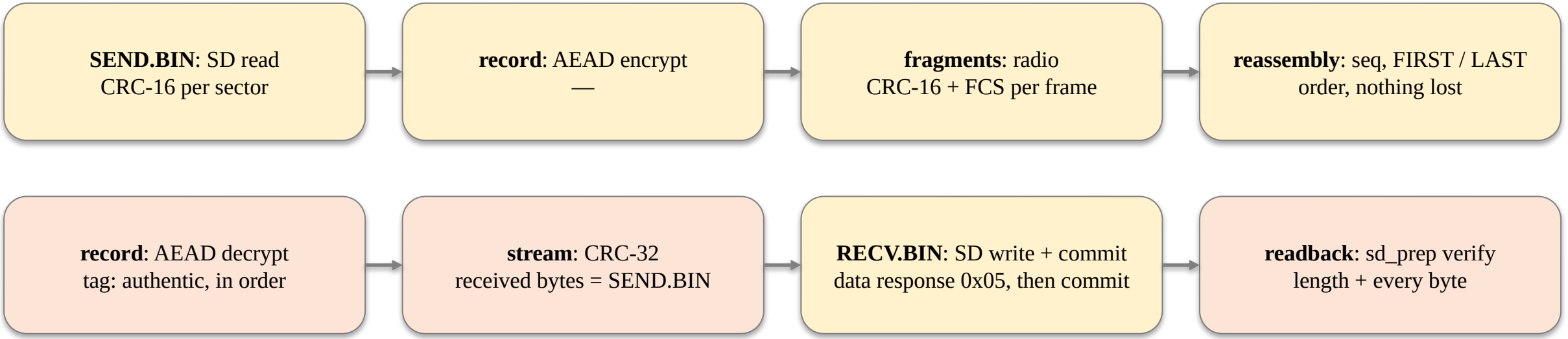
4. End-to-End Transfer (2/8) From file bytes to radio frames



1 MiB, card A → card B	Count
SD sectors read on A / written on B	2,048 / 2,048
records: 5,090 × 206 B + 1 × 36 B, plus the length and CRC records	5,093
data frames ('D'), and as many 'A' frames back	10,183
time per data frame: 99.6 s / 10,183	≈ 9.8 ms

- Records do not line up with sectors (512 = 2 × 206 + 100): fat32_read copies across sector borders, and the one-sector buffer still reads each sector once.
- No buffer grows with the file: one record, a few frames and one sector, the same for 1 KB or 4 GB.

4. End-to-End Transfer (3/8) A check at every step



Console line	Proves	Does not prove
XFER SENT (board A)	A read its file and every record was accepted by the link	anything about B's card
… CRC cffbfb2e OK (board B)	B received exactly the bytes of SEND.BIN	that they were stored
XFER RECV … -> RECV.BIN	B also committed RECV.BIN	that the card holds them correctly
sd_prep: <size sent> bytes, 0 bytes differ	the stored file, read back, is right (generated SEND.BIN only)	—

4. End-to-End Transfer (4/8) Build and run

```
Run this
cd $REPO/fpga/sw/export/edhoc-authkem
make edhoc_authkem_initiator edhoc_authkem_responder SUITE=TBD3 EXTRA_CFLAGS=-DAPP_SD_XFER
../common/board/run_pair.sh xfer build-tbd3/edhoc_authkem_initiator.elf \
                                build-tbd3/edhoc_authkem_responder.elf 1500
# then check board B's card (generated SEND.BIN of 1 MiB):
cd ../tests && make sd_prep EXTRA_CFLAGS=-DSD_PREP_VERIFY=1048576
BOARD=responder ../common/board/run_one.sh build/sd_prep.elf 20
```

SEND.BIN came from	How to verify RECV.BIN
sd_prep (generated test pattern)	sd_prep -DSD_PREP_VERIFY=<bytes>: the byte count must equal the size sent, CRC as sent, 0 bytes differ
your own file (PDF, image …)	copy RECV.BIN to a PC; compare size and sha256sum with the original — sd_prep cannot check it

- “0 bytes differ” alone is not enough: a shorter file compares equal over its own length. Check the reported byte count too.
- Remove the old RECV.BIN first (on a PC, or reformat card B: sd_prep -DSD_PREP_FORMAT), or a failed run leaves the old, identical file in place.

4. End-to-End Transfer (5/8) Real run: 1 MiB from card A to card B

```
board A (initiator)
XFER: sending SEND.BIN, 1048576 bytes
PERF xfer_send    7465350439 cyc 99.538005 s 1048576 bytes 10534 B/s
PERF xfer_sd_part      427826394 cyc 5.704351 s
XFER SENT: 1048576 bytes, CRC cffbfb2e
```

```
board B (responder)
XFER: receiving 1048576 bytes
PERF xfer_recv    7469254102 cyc 99.590054 s 1048576 bytes 10528 B/s
PERF xfer_sd_part      490423540 cyc 6.538980 s
XFER RECV: 1048576 bytes -> RECV.BIN, CRC cffbfb2e OK
```

```
board B, sd_prep verify
card: 7569408 sectors (3696 MiB)
RECV.BIN: 1048576 bytes, CRC cffbfb2e, 0 bytes differ
SD PREP OK
```

- **1 MiB in 99.6 s = 10.5 KB/s**, post-quantum TBD3 keys, AES-256-GCM.
- SD time: 5.7 s read + 6.5 s write — the radio sets the pace.

4. End-to-End Transfer (6/8) When something is missing

```
board A, card without FAT32
XFER: SD card not usable (-2)
XFER FAIL 3
```

```
board B
XFER: peer has no SEND.BIN
XFER FAIL 3
```

- The sender announces “no file” (length 0xFFFFFFFF) instead of going silent: both boards stop cleanly in 3 s, no 5-minute timeout.
- If board B's card fails, it still receives and checks the whole stream (CRC) and reports “not stored” — both sides stay in step.
- A wrong CRC or a broken record: RECV.BIN is not written, the old one stays.

4. End-to-End Transfer (7/8) If board B says “not stored”

```
Example output — board A
XFER: sending SEND.BIN, 1048576 bytes
XFER SENT: 1048576 bytes, CRC cffbfb2e
APP DATA OK
```

```
Example output — board B, same run
XFER: receiving 1048576 bytes
SD: write 84310: data response 1f      (three tries)
XFER: SD write failed at 618 (-1)
XFER: received 1048576 bytes, CRC cffbfb2e OK, not stored (-1)
XFER FAIL 3
```

- The link worked: handshake, every record and the transfer CRC are fine. **Storage failed**: the first sector write of RECV.BIN was refused three times, so B kept receiving and checking but did not store.
- Board A's “XFER SENT … APP DATA OK” is true and still not success: judge a transfer by board B's last lines, then read the file back (section 4, (4/8)). run_pair.sh reports this run as FAIL (no-APP-OK on B).
- What the SD: reason means and what to do: section 3, (6/9).
- Remove the old RECV.BIN before a run (PC, or reformat card B), or a failed run leaves the previous, identical file and the read-back still says 0 bytes differ.

4. End-to-End Transfer (8/8) Larger files: how to run the experiment

Step	How	Watch out
1. Make the file	copy it as SEND.BIN with a card reader, or sd_prep -DSD_PREP_SEND=<bytes>	up to 4,294,967,294 B
2. Room on card B	free space ≥ file size, in **one contiguous run** (section 3)	else “no room” (FAT_ENOSPC): reformat B
3. Timeout	4th argument of run_pair.sh: size / 10 KB/s + 60 s	1 MiB ≈ 100 s → 200 s is safe
4. Run, keep logs	LOGDIR=xfer_runs ../common/board/run_pair.sh …	one log pair per run
5. Verify	generated file: sd_prep -DSD_PREP_VERIFY=<bytes>; own file: size + sha256sum on a PC	byte count = size sent
6. Compare	PERF xfer_send / xfer_recv B/s and the “STACK whole run” line per size	peak should not grow (64 KiB and 1 MiB: equal)

- Units here: 1 MiB = 1,048,576 B; KB/s = 1,000 B/s; PERF prints B/s. Measured: 1 MiB in 99.6 s. Anything larger in these slides is an **estimate** from that rate until you measure it.

Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. **Speed: Measure and Choose a Link**
6. Choose and Reproduce
7. Practice

5. Speed: Measure and Choose a Link (1/4) Measure the speed yourself

```
Run this (add TRANSPORT=uart for the wired link)
cd $REPO/fpga/sw/export/edhoc-authkem
make edhoc_authkem_initiator edhoc_authkem_responder SUITE=TBD3 EXTRA_CFLAGS=-DAPP_SD_XFER
../common/board/run_pair.sh xfer build-tbd3/edhoc_authkem_initiator.elf \
                                build-tbd3/edhoc_authkem_responder.elf 300
grep -h "PERF xfer\|PERF handshake_total" boardlogs/xfer.*.log
```

TBD3, measured on the boards	Radio (default)	UART1, 921,600 baud
Handshake (PERF handshake_total, initiator)	1.25 s	0.77 s
1 MiB, SD card A → SD card B (PERF xfer_recv)	99.6 s, 10.5 KB/s	32.2 s, 32.6 KB/s
256 KiB generated data (APP_FILE_BYTES=262144)	—	49.5 KB/s

- PERF xfer_send / xfer_recv: time from the first to the last file record, SD access included (xfer_sd_part shows the SD share). Units: KB/s = 1,000 B/s.
- Your numbers should be within a few %; the radio varies more (interference on channel 11).

5. Speed: Measure and Choose a Link (2/4) Where the radio time goes

Per radio frame	Time
full frame on air (116 B payload)	6.3 ms
tiny frame (5 B payload)	2.5 ms
→ fixed cost inside the radio chip	≈ 1.8 ms
→ per byte at 250 kbit/s	32 µs
software per frame (AES, CRC, SPI)	≈ 1 ms

Radio setting in this code	Value
channel	11
bit rate	250 kbit/s (turbo mode loses frames)
radio SPI clock	4.7 MHz (9.4 MHz gives bit errors)
CSMA back-off	off (only two nodes)
record size	206 B = two full frames

- A data frame costs ≈ 9.8 ms (1 MiB = 10,183 frames in 99.6 s); the radio chip sets the pace, not the software.
- So the way to go faster is a faster link — (4/4).
- Compare your runs with a control run made at the same time: channel 11 has interference spells.

5. Speed: Measure and Choose a Link (3/4) Fast crypto on this core

Measured in this code, 75 MHz	Cycles
AES-256, one block (T-table)	2,224
GHASH, one block multiply	3,326
AES-256-GCM, 192-byte record	91,741

Simplified — common/crypto/aes_gcm.c

```
static inline uint64_t top_mask(uint64_t x) { return (uint64_t)((int64_t)x >> 63); }
uint64_t m = top_mask(xh);   // all ones if the top bit is set: a shift, no multiply
zh ^= vh & m;
```

- This core's multiplier is **bit-serial: ~64 cycles per mul**. GCC turns a masked select like `(0 - bit) & v` into a multiply, so the hot loops build their masks with shifts instead. Check a build: `riscv64-unknown-elf-objdump -d <elf> | grep -c mul`.
- AES uses the 32-bit T-table form: one 1 KB table built at first use, 16 look-ups per round.
- See it on your board: after a 1 MiB file transfer, the `PERF encrypt` / `PERF decrypt` lines show about 160 KB/s for AES-256-GCM — far above the link speed (short echo records alone show less).

5. Speed: Measure and Choose a Link (4/4) A wired link: UART1 at 921,600 baud

TBD3, measured on the boards	Radio (MRF24J40)	UART1, 921,600 baud
Handshake	1.25 s	0.77 s
1 MiB, SD card A → SD card B	99.6 s (10.5 KB/s)	32.2 s (32.6 KB/s)

- Wiring: A's JB2 (TX) → B's JB1 (RX), B's JB2 (TX) → A's JB1 (RX), JB5 GND ↔ JB5 GND (Lesson 1, section 4).
- Build both images with `TRANSPORT=uart`; `EXTRA_CFLAGS=-DUART1_BAUD=<baud>` changes the speed (both boards alike).
- Flow control (*edhoc_transport.c*): while a board waits for a frame it sends a READY byte every 0.2 ms; the sender waits for a fresh READY, then sends the whole frame at line rate. A receiver busy with AES or the SD card cannot lose bytes.
- Card to card, the SD card now takes a visible share: about 5.7 s of the 32.2 s on each board.

Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Speed: Measure and Choose a Link
6. **Choose and Reproduce**
7. Practice

6. Choose and Reproduce (1/3) Choose a configuration

If you want …	Build / run	Lesson
the smallest and fastest classical handshake	edhoc-wolfssl, METHOD 3, CRYPTO_SUITE=0 or 4	3
certificates / signatures	methods 0–2, e.g. edhoc_m0_* CRYPTO_SUITE=2	3
a byte-exact check against the standard	make rfc9529-test	3
post-quantum key exchange	edhoc-authkem, SUITE=TBD2 (default) → build/	4
AES-256 + ML-KEM-1024 together	edhoc-authkem, SUITE=TBD3 → build-tbd3/	4
to move a file card to card	EXTRA_CFLAGS=-DAPP_SD_XFER	5
a faster link than the radio	TRANSPORT=uart, 3 wires on JB	5
time per message step / every value	TIMING=1 / DEBUG=1 (not both for timing)	3, 4

Every row runs on the same bitstream; only the firmware build changes.

6. Choose and Reproduce (2/3) Limits to know before you reuse this code

Limit	Why	What to do
Random numbers (RFC 9528 §9.2)	this SoC has no TRNG; demo seeds come from the cycle counter	connect edhoc_platform_entropy() to real entropy
Post-quantum drafts	TBD1–3 unassigned; draft -01 may change	labels: one define; a new revision: full review + reference model
Radio speed	250 kbit/s chip, ~10.5 KB/s end to end; 100 MiB ≈ 2.8 h (estimate)	TRANSPORT=uart over 3 wires on PMOD JB: 32.6 KB/s card to card
SD storage	commit is not power-loss safe; RECV.BIN needs contiguous free space	verify by readback; fsck.fat after a crash

6. Choose and Reproduce (3/3) Reproduce everything

Step	Lesson	You should see
1. Set up (Lesson 1, section 1, (6/12)–(10/12)), build the bitstream, program both boards	1	timing met, jtag_test PASS
2. Toolchain, peripheral + crypto tests	2	every test PASS
3. EDHOC pairs, RFC 9529 test	3	PASS per pair, RFC9529 PASS
4. AuthKEM TBD1–3, reference model	4	PASS (prk=match with DEBUG=1), REFERENCE MODEL: PASS
5. Prepare cards, transfer, verify RECV.BIN	5	XFER RECV … OK, then 0 bytes differ

- Every number in these lessons comes from such runs; the log format (PERF / STACK lines) and the scripts (run_pair.sh, perf_table.py) let you compare yours line by line.
- `VERBOSE=1 run_pair.sh ...` shows both consoles live; a DEBUG=1 build adds every intermediate value.

Outline

1. Application Channel
2. Radio Link
3. SD Card & FAT32
4. End-to-End Transfer
5. Speed: Measure and Choose a Link
6. Choose and Reproduce
7. **Practice**

7. Practice (1/3) Exercises

Exercise 1:

- Send a real file (a PDF, an image) from card A to card B with TBD3. Compare the two files on a PC (sha256sum).

Exercise 2:

- Build the radio link without the CRC (remove `seal` and the check). Stream 1 MiB. What fails, after how long? Restore the file afterwards.

Exercise 3:

- Measure KB/s for 64 KiB, 256 KiB and 1 MiB. Is the speed constant? Estimate the time for 100 MiB.

7. Practice (2/3) What to put in the report

- Exercise 1: both consoles, the two sha256 values, the time and KB/s.
- Exercise 2: the console at the failure and your explanation.
- Exercise 3: a table and a chart of size vs. time; your 100 MiB estimate.

7. **Practice** (3/3) Answer notes

Exercise	What you should see	Why
1. Real file	identical sha256 on both files; ≈ 100 s per MiB plus a few seconds of handshake	CRC-32 and AEAD protect the path; the radio sets the pace
2. No CRC	sooner or later an AEAD failure on B (record at <n> failed (113)) — or, on a lucky run, none	a flipped SPI bit is no longer caught before the AEAD sees it
3. Sizes	about the same B/s for every size	the cost is per frame; nothing in RAM grows with the file

- Restore after exercise 2: from *fpga/sw/export*, `git checkout common/src/edhoc_rf_link.c`, rebuild both images, rerun: XFER RECV … OK.
- Negative cases to recognise: no SEND.BIN → both stop with XFER FAIL 3 in seconds; B's card fails → B says “not stored” while A says SENT (section 4, (7/8)); a broken record → B rejects it and keeps the old RECV.BIN.

Today's lesson highlight:

- PRK_out → EDHOC_Exporter → K_app / IV_app; 206-byte AEAD records, nonce from direction + sequence number.
- The radio link checks every frame (CRC-16), re-initialises a stuck radio after 2 s and keeps listening while it repeats; the RF: line shows all three.
- FAT32 in one 512-byte buffer: SEND.BIN → RECV.BIN, committed only when the CRC matches.
- 1 MiB card to card with TBD3: 99.6 s by radio, 32.2 s over 3 UART wires; verify with the byte count and the read-back.
- A transfer succeeded only when board B says RECV … OK **and** the read-back matches — the sender's "SENT" is not enough.
- Before real use: a real entropy source, and final post-quantum code points.

End of the course

Thank you for following all five lessons

国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

THANK YOU

2026/12